

QL-ILS: Integrando Q-Learning e Iterated Local Search para o TSP

Gabriel Souto, Leonardo Assumpção, Lucas Barbosa,
Matheus Bastos, Rafael Dias

PESC/COPPE/UFRJ – OTIMIA (COS887)

09/12/2025

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Metodologia
- 5 Experimentos
- 6 Conclusões Esperadas

Motivação

- O **TSP** (Traveling Salesman Problem) é NP-difícil com vasta aplicação prática: logística, roteamento, manufatura.
- **Heurísticas construtivas** geram soluções iniciais rapidamente.
- **Buscas locais** refinam essas soluções.
- **Meta-heurísticas** como ILS combinam perturbações e buscas locais para escapar de ótimos locais.

Pergunta central: Podemos usar **Reinforcement Learning** para aprender *qual combinação* de estratégias funciona melhor em cada situação?

Objetivos

- 1 Propor uma abordagem híbrida **QL-ILS** que integra Q-Learning ao framework do Iterated Local Search.
- 2 **Aprender**, a partir de execuções em instâncias de treino, qual combinação (perturbação + busca local) tende a melhorar soluções.
- 3 **Avaliar** se a política aprendida generaliza para instâncias não vistas.

Foco em reduzir o *gap* (em relação à melhor solução conhecida) de forma automática, sem ajuste manual de parâmetros.

Sumário

- 1 Introdução
- 2 Fundamentos**
- 3 Modelagem MDP
- 4 Metodologia
- 5 Experimentos
- 6 Conclusões Esperadas

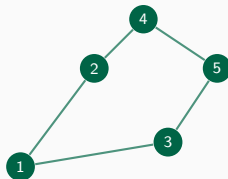
O Problema: TSP

Traveling Salesman Problem:

- Dado um conjunto de n cidades e distâncias d_{ij} , encontrar o **tour de menor custo** que visite todas as cidades exatamente uma vez.

$$\min_{\pi} \sum_{i=1}^n d_{\pi(i), \pi(i+1)} \quad (\pi(n+1) = \pi(1))$$

- Complexidade: **NP-difícil**
- Soluções exatas: impraticáveis para n grande
- Alternativa: heurísticas e meta-heurísticas



Exemplo: tour com
5 cidades

Iterated Local Search (ILS)

Estrutura básica:

- ① Gerar solução inicial s_0
- ② Aplicar busca local:
 $s \leftarrow \text{BL}(s_0)$
- ③ Repetir:
 - Perturbar: $s' \leftarrow \text{Pert}(s)$
 - Refinar: $s'' \leftarrow \text{BL}(s')$
 - Aceitar se melhora
- ④ Retornar melhor solução

Componentes intercambiáveis:

Diferentes heurísticas construtivas e buscas locais podem ser combinadas.

Como escolher a melhor combinação?

Componentes do QL-ILS

Heurísticas Construtivas:

- **Random**: tour aleatório
- **Nearest Neighbor**: cidade mais próxima
- **Cheapest Insertion**: menor aumento de custo

Buscas Locais:

- **2-opt**: troca 2 arestas; $O(n^2)$
- **Lin-Kernighan**: cadeias de 2-opt; mais potente

Perturbações:

- Troca de dois vértices
- Reversão de segmento

8 combinações de ações:
(perturbação ou construtivo) $\times$ busca local

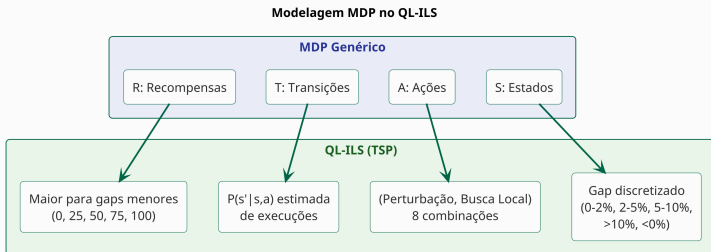
Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP**
- 4 Metodologia
- 5 Experimentos
- 6 Conclusões Esperadas

Modelagem como MDP

MDP: Processo de Decisão de Markov (S, A, T, R)

- **Estado** s : gap percentual discretizado
- **Ação** a : (perturbação, busca local)
- **Transição** T : estimada empiricamente
- **Recompensa** R : maior para gaps menores



Discretização de Estados

Gap (%)	Estado	Recompensa	Interpretação
$0 \leq \text{gap} \leq 2$	0	75	Excelente
$2 < \text{gap} \leq 5$	1	50	Bom
$5 < \text{gap} \leq 10$	2	25	Regular
$\text{gap} > 10$	3	0	Ruim
$\text{gap} < 0$	4	100	Melhor que bfs

$$\text{gap} = \frac{\text{custo}_{\text{atual}} - \text{custo}_{\text{bfs}}}{\text{custo}_{\text{bfs}}} \times 100\%$$

Espaço de Ações

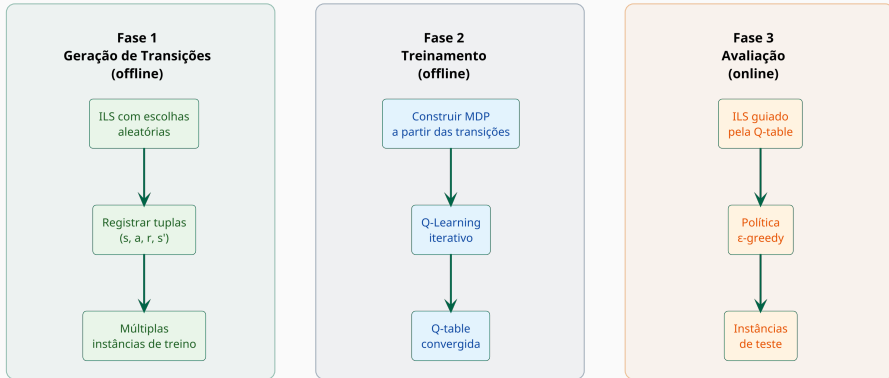
Ação	Perturbação	Busca Local
0	Two Swap	2-opt
1	Two Swap	Lin-Kernighan
2	Segment Reverse	2-opt
3	Segment Reverse	Lin-Kernighan
4	Random (construtivo)	2-opt
5	Nearest (construtivo)	2-opt
6	Cheapest (construtivo)	2-opt
7	Nearest (construtivo)	Lin-Kernighan

A **Q-table** mapeia: estado $\rightarrow$ valor de cada ação

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Metodologia**
- 5 Experimentos
- 6 Conclusões Esperadas

Fluxo do QL-ILS

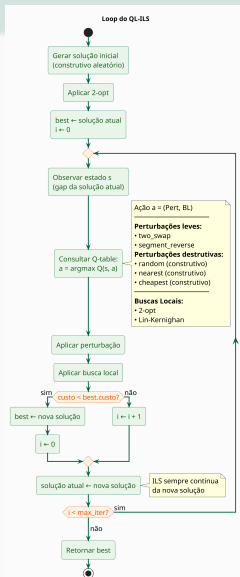


Separação treino/teste: Q-table treinada offline; aplicada rapidamente em novas instâncias.

Loop Principal do QL-ILS

Execução guiada por Q-table:

- 1 Gerar solução inicial (2-opt)
- 2 Loop:
 - Observar estado s (gap)
 - Consultar Q-table:
 $a = \arg \max Q(s, a)$
 - Executar ação a (pert + BL)
 - Se melhorou: atualizar best
 - Sempre: continuar da nova solução
- 3 Retornar melhor encontrada



Algoritmos de Aprendizado

Single Q-Learning (iteração de valor):

$$Q(s, a) \leftarrow R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q(s', a')$$

Double Q-Learning:

- Mantém duas Q-tables (Q_1, Q_2)
- Q_1 usa $\max_{a'} Q_2(s', a')$ para update (e vice-versa)
- Reduz **superestimação** do valor Q

Implementação *model-based*:

MDP estimado das transições; convergência garantida.

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Metodologia
- 5 Experimentos**
- 6 Conclusões Esperadas

Procedimento Experimental

Instâncias:

- Tipos: EUC_2D, ATT, GEO
- Tamanhos: 10, 20, ..., 100 vértices
- ~1111 por combinação
- Split: 70% treino, 30% teste

Métricas:

- Gap percentual ao bfs
- Tempo de execução
- Variância entre repetições

Ambiente:

- Python + PyTorch
- Q-tables por tamanho

Ambiente Computacional

Arquitetura de execução:

- Ambiente **distribuído** com 3 máquinas
- Total de **50 núcleos** operando em paralelo
- Recursos heterogêneos para escalar experimentos

Gerenciamento de recursos:

- Cada máquina operou com $n - 2$ núcleos ativos
- Evitamos sobrecarga e reduzimos risco de falhas em execuções longas

Especificações:

- (i) AMD Ryzen, 16 núcleos
- (ii) 28 núcleos
- (iii) 12 núcleos

Motivação:

Viabilizar prazos de execução para ~1111 instâncias por configuração (métrica, tamanho) para um total de 33330

Resultados: Exemplos

Instância	Nós	Gap	Tempo (s)
random_564	10	0.00%	0.06
random_2545	30	0.00%	7.2
random_3284	30	0.00%	12.2
random_2681	30	0.47%	12.6
random_7094	70	0.96%	278.1
random_6920	70	1.57%	110.7
random_7184	70	3.19%	130.9
random_7824	80	4.95%	264.3
random_9879	90	5.40%	448.6

Tipo: EUC_2D. Gap médio cresce com o tamanho da instância.

Análise dos Resultados

Observações:

- Instâncias pequenas ($n = 10-30$):
frequentemente **gap = 0%**
- Instâncias maiores ($n = 70-100$):
gaps tipicamente 1-6%
- Q-table generaliza para
instâncias não vistas
- Tempo:
0.06s ($n = 10$) a 450s ($n = 90$)

Limitações:

- Requer bfs conhecido no treino
- Discretização simplista
- Tempo cresce com n

Sumário

- 1 Introdução
- 2 Fundamentos
- 3 Modelagem MDP
- 4 Metodologia
- 5 Experimentos
- 6 Conclusões Esperadas**

Discussão

Vantagens do QL-ILS:

- Aprende automaticamente qual estratégia usar
- Sem ajuste manual de parâmetros
- Q-table compacta e interpretável

Desafios:

- Dependência do bfs conhecido
- Balanceamento ϵ
(diversificação / intensificação)
- Generalização *cross-domain*

Double Q-Learning vs Single:

Esperamos double mais robusto (evita superestimação), resultados finais similares em gap.

Conclusões Esperadas

- **QL-ILS** é viável para integrar RL a meta-heurísticas para o TSP.
- Política aprendida seleciona estratégias adequadas, com gaps competitivos.
- Mais eficaz para instâncias pequenas/médias; '*grandes*' ainda apresentam gaps significativos.
- Separação **treino (offline)** / **execução (online)** permite aplicação rápida.

Trabalhos Futuros

- Representação de estado mais rica (features além do gap)
- Redes neurais para aproximar Q (**Deep Q-Learning**)
- Funções de recompensa alternativas ($R = -\Delta_{\text{custo}}$)
- Generalização entre tipos de distância
- Comparação com outras meta-heurísticas híbridas

Referências

- Wang, J. et al. (2023). *Reinforcement learning for the traveling salesman problem*. The Journal of Engineering.
- Bengio, Y., Lodi, A., Prouvost, A. (2020). *Machine Learning for Combinatorial Optimization*. arXiv:1811.06128.
- Sutton, R. S., Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.

Agradecimentos

Agradecemos à turma e às professoras
Priscila Lima e **Laura Bahiense**
pela atenção e oportunidade de aprendizado.

Equipe:

Gabriel Souto · Leonardo Assumpção · Lucas Barbosa
Matheus Bastos · Rafael Dias

Obrigado!